

MANJARA CHARITABLE TRUST
RAJIV GANDHI INSTITUTE OF TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

Experiment No: 11

Name: Dnyaneshwari Kadam
Roll No: A3-441

Date:

Aim: Case study of windows operating system

1. Introduction

The Microsoft Windows family of operating systems has been a dominant force in personal and commercial computing since the 1990s. This case study focuses on Windows XP, a landmark release that successfully unified Microsoft's consumer (Windows 95/98/Me) and professional (Windows NT/2000) lines. As detailed in *Operating System Concepts* (Silberschatz, Galvin, Gagne), Windows XP was built upon the robust architecture of Windows NT, incorporating a preemptive multitasking kernel, advanced security features, and broad hardware support. It serves as an excellent example of a modern, monolithic, client-server operating system designed for extensibility, portability, and performance.

2. History

The development of modern Windows began in 1988 when Microsoft hired Dave Cutler, the architect of DEC's VAX/VMS, to build a "new technology" (NT) portable operating system. Originally intended to use the OS/2 API, the project shifted to use the 32-bit Windows API (Win32) due to the growing popularity of Windows 3.0. The first versions, Windows NT 3.1 and 3.5, established the system's core architecture.

- **Windows NT 4.0 (1996):** A major update that moved the graphical user interface (GUI) routines and graphics code into the kernel to improve performance, though it slightly decreased system reliability.
- **Windows 2000 (2000):** Released as an update to NT 4.0, it added Active Directory (an X.500-based directory service), better networking and laptop support, plugandplay, and a distributed file system.

Windows XP (2001): This was a significant milestone, intended as both an update to Windows 2000 for business users and a replacement for Windows 95/98 for home users. It introduced a new visual design, improved networking and device support (including wireless), automatic system repair features, and better performance and security. For the first time, a 64-bit version was also released.

Design Principles

The design of Windows XP (and its predecessors) was guided by several key goals, as outlined in the source text:

- **Security:** Beyond standard C-2 security classification, Windows XP incorporated extensive code review, automatic analysis tools, and a "compatibility layer" to make the system look bug-for-bug compatible with older versions, thus reducing application crashes and security holes.
- **Reliability:** The system was built for stability. Features included driver verification to catch subtle bugs, a certification process for third-party drivers and applications, and system health monitoring that could download fixes before problems were encountered.
- **Windows and POSIX Application Compatibility:** A primary goal was to run a vast range of existing applications. This was achieved using a **compatibility layer** that emulates the quirks of older Windows versions, a **thunking layer** to translate 16-bit API calls to 32-bit, and a new POSIX subsystem called **Interix** for better UNIX compatibility.
- **High Performance:** Designed for performance on desktops (I/O-bound), servers (CPU-bound), and multiprocessor systems, Windows XP used techniques like asynchronous I/O, optimized network protocols, kernel-based graphics, sophisticated caching, and advanced, scalable locking protocols (e.g., queued spinlocks) for multiprocessor systems.
- **Extensibility & Portability:** A layered architecture allowed for easy addition of new features, file systems, and device drivers. Most of the operating system is written in C and C++, with processor-dependent code isolated in the **Hardware Abstraction Layer (HAL)**, making the system portable to different hardware architectures (IA32 and IA64).
- **International Support:** Windows XP was designed for global use. It uses **UNICODE** as its native character code, provides a **National Language Support (NLS) API** for locale-specific formatting (dates, time, currency), and keeps text strings in resource files for easy localization.

System Components (Architecture)

Windows XP uses a layered, client-server architecture. The main layers are the Hardware Abstraction Layer (HAL), the kernel, and the executive, all of which run in **kernel mode** (protected). A collection of subsystems and services run in **user mode**.

- **Hardware-Abstraction Layer (HAL):** The HAL hides hardware differences from the upper layers of the OS. It provides a virtual machine interface used by the kernel, executive, and device drivers. This allows a single driver version to run on all

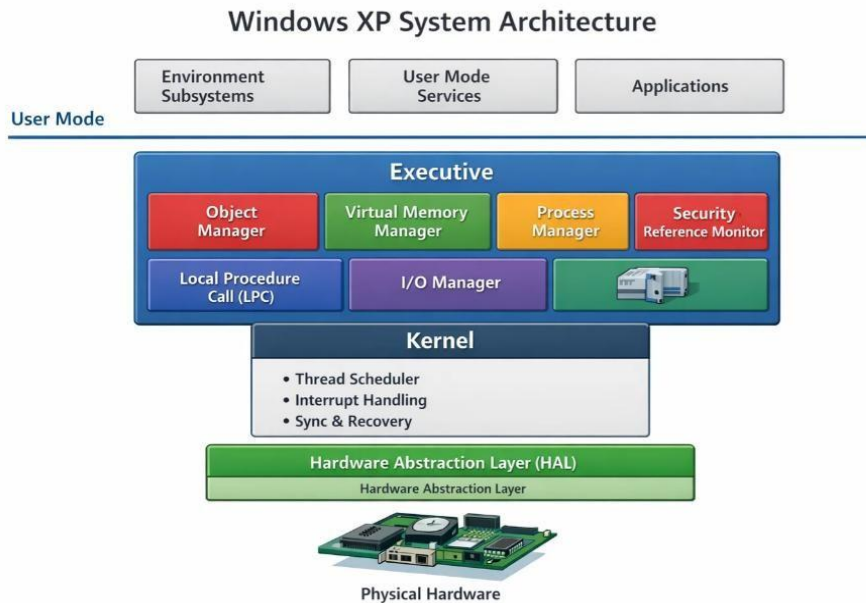
supported hardware platforms and provides support for symmetric multiprocessing (SMP).

- **Kernel:** The kernel has four main responsibilities: thread scheduling, interrupt and exception handling, low-level processor synchronization, and recovery after power failure. It uses **dispatcher objects** (events, mutexes, semaphores, timers) for synchronization. Scheduling is preemptive and priority-based, using a 32-level scheme (0-31). Priorities 16-31 are for real-time threads, and 1-15 for variable priority threads. The dispatcher uses a queue for each priority level.
- **Executive:** The executive provides a core set of services used by all environmental subsystems. Key components include:

- **Object Manager:** Manages all kernel-mode entities as objects, providing a generic interface for creating, naming, and securing them. It maintains an internal name space and uses **handles** for user-mode code to access objects.
- **Virtual Memory Manager:** Manages virtual address spaces, physical memory allocation, and paging. On IA32 processors, each process has a 4-GB virtual address space (upper 2 GB for kernel-mode, lower 2 GB for usermode). It uses a two-level page table structure (page directory and page tables) and a working-set model for memory management.
- **Process Manager:** Handles the creation and deletion of processes, threads, and jobs. It provides services for managing process and thread priorities, affinities, and for inter-process communication via asynchronous procedure calls (APCs).
- **Local Procedure Call (LPC) Facility:** An optimized, message-passing mechanism for communication between client and server processes on the *same* machine. It is used extensively by environmental subsystems and can use port message queues, shared memory sections, or direct API calls to pass data.
- **I/O Manager:** Manages all file systems, device drivers, and network drivers.

It converts I/O requests into a standard format called an **I/O Request Packet (IRP)** and forwards it through a driver stack. It also works with the **Cache Manager** to provide caching for the entire I/O system.

Security Reference Monitor (SRM): The central authority for run-time access validation and audit checking. When a process opens a handle to an object, the SRM checks the process's security token against the object's access-control list (ACL) to determine if access should be granted.

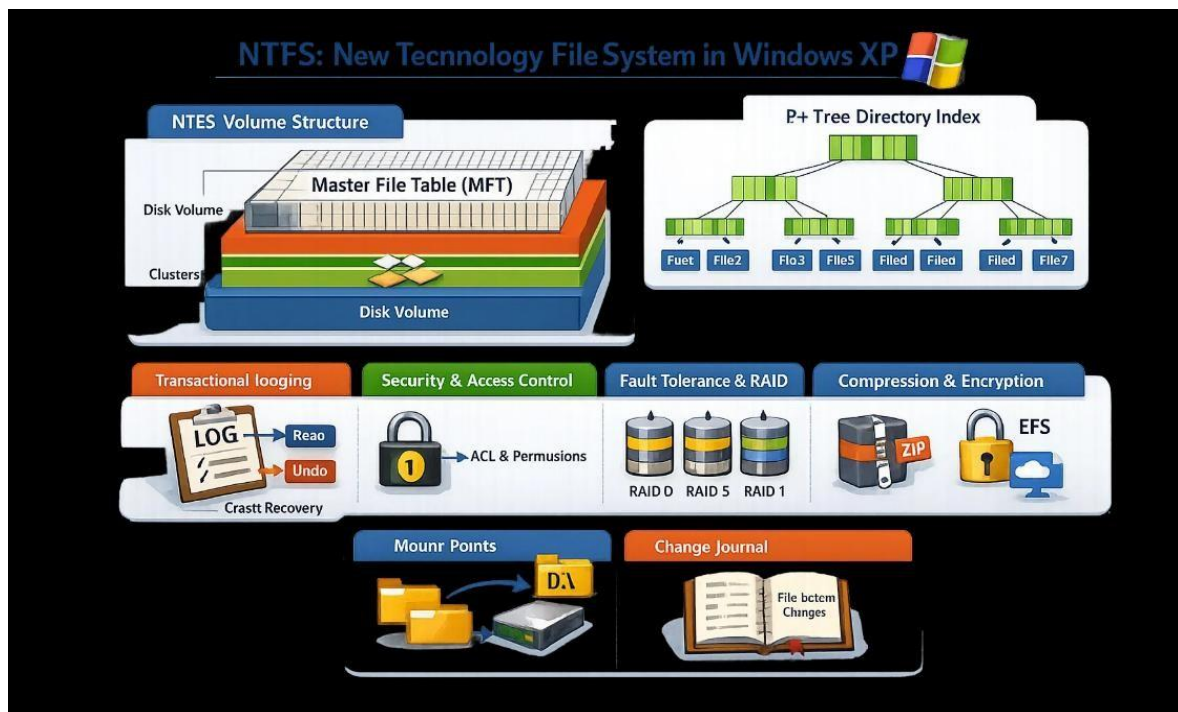


File System (NTFS)

Windows XP uses **NTFS (New Technology File System)** as its primary file system. It was designed to overcome the limitations of the older FAT (File Allocation Table) systems.

- **Internal Layout:** The fundamental entity in NTFS is a **volume**, which can be a portion of a disk, an entire disk, or span multiple disks. NTFS uses **clusters** as the unit of disk allocation (default size from 512 bytes to 4 KB, depending on volume size).
- **Master File Table (MFT):** Every file and directory on an NTFS volume is described by one or more records in an array stored in a special file called the **Master File Table (MFT)**. Small attributes (e.g., file name, timestamps) are stored in the MFT record itself (resident). Large attributes (e.g., file data) are stored in **extents** (contiguous runs of clusters on disk) outside the MFT, with pointers to them stored in the MFT record.
- **B+ Tree:** NTFS directories use a **B+ tree** data structure to store an index of file names. This provides efficient searching, even for very large directories. The directory entry contains the file name, file reference, and a copy of the update timestamp and file size for quick directory listing.
- **Recovery:** NTFS uses **transaction-based logging** to ensure file-system consistency. Before a metadata structure is altered, a log record with redo and undo information is written. After a crash, the system can replay the log to commit successful transactions and undo incomplete ones, ensuring metadata is always consistent.
- **Other Features:**

- **Security:** Each file references a **security descriptor** containing the owner's access token and an **Access Control List (ACL)** defining user privileges.
- **Fault Tolerance:** Supports volume sets (concatenating partitions), stripe sets (RAID 0), stripe sets with parity (RAID 5), and disk mirroring (RAID 1) via the FtDisk driver.
- **Compression & Encryption:** Supports data compression on a per-file or perdirectory basis and file-level encryption using the Encrypting File System (EFS).
- **Mount Points & Change Journal:** Allows a volume to be mounted on an empty directory (mount point) and keeps a journal of all file-system changes for services like content-indexing and file-replication.



Conclusion

The Windows XP operating system exemplifies how core theoretical concepts—process management, memory virtualization, file systems, and security—are integrated into a practical, high-performance, and reliable commercial system. Its layered architecture, centered on a privileged kernel and executive, successfully balances the competing demands of application compatibility (via environmental subsystems), hardware abstraction (via the HAL), and system protection (via the Security Reference Monitor). Key features like the unified page cache for efficient I/O, the transactional NTFS for data integrity, and a modular driver model for extensibility demonstrate that a modern OS must be a cohesive synthesis of many specialized components. Ultimately, Windows XP's success lies not in any single innovation, but in the robust and pragmatic engineering of its entire structure to meet the complex needs of real-world computing.